

Overwatch Patch Impact Analytics Pipeline

End-to-End ETL + Causal Inference Project Plan (for Activision Blizzard Internship Application)

Rachel Sooyeon Lee

2026 년 8 월

Contents

0. 프로젝트 개요 (Overview)	2
1. 최종 리포지토리 폴더 구조	2
2. 아키텍처 흐름	4
3. 환경 설정	4
3.1 requirements.txt	4
3.2 .env.example	5
4. Extract 단계	5
4.1 extract/overfast_client.py	5
4.2 extract/patch_notes_scraper.py	7
4.3 extract/kaggle_loader.py	8
5. Load 단계	9
5.1 load/bigquery_loader.py	9
6. Transform 단계 (dbt)	10
6.1 transform/dbt_project.yml	10
6.2 transform/models/staging/stg_hero_stats.sql	10
6.3 transform/models/staging/stg_patch_notes.sql	11
6.4 transform/models/staging/schema.yml (dbt tests)	11
6.5 transform/models/marts/mart_patch_event_panel.sql	12
6.6 transform/models/marts/mart_hero_stats_daily.sql	13
6.7 카운터 관계 매핑 (Counter Relationships)	14
6.7.1 transform/seeds/counter_relationships.csv	14
6.7.2 transform/models/marts/mart_counter_spillover.sql	14
6.7.3 analysis/counter_spillover_analysis.py	15
7. Analysis 단계 (Causal Inference)	16
7.1 analysis/did_analysis.py	16
7.2 analysis/event_study.py	18
8. Orchestration 단계 (Airflow)	19
8.1 orchestration/dags/patch_impact_dag.py	19
8.2 docker-compose.yml (요약)	20

9. Dashboard 단계	21
9.1 dashboard/app.py	21
10. CI/CD	22
10.1 .github/workflows/ci.yml	22
10.2 tests/test_overfast_client.py (예시 유닛 테스트)	23
11. Dockerfile	24
12. 배포 (Deployment) 단계	24
13. README.md 초안 (GitHub 용)	25
14. 진행 체크리스트 & 예상 소요 기간	25
15. 레쥬메 Bullet 초안	26
16. 결과물 요약 (TL;DR)	26

0. 프로젝트 개요 (Overview)

목적: Overwatch 패치 (buff/nerf/rework) 가 히어로의 win rate / pick rate 에 실제로 통계적으로 유의미한 **인과 효과 (causal effect)** 를 주는지, 그리고 counter/synergy 관계에 있는 다른 히어로들에게 **spillover effect** 가 있는지를 자동으로 추적·분석하는 end-to-end 데이터 파이프라인.

왜 이 프로젝트인가:

- Activision Blizzard 의 Ed Tien 이 언급한 두 가지 기준을 정확히 겨냥: (1) causal inference 중심 워크, (2) 게임/엔터테인먼트 도메인 프로젝트 (과거 인턴들의 LoL 인게임 화폐 분석, WoW Corrupted Blood 사례처럼)
- 기존 레쥬메 (CHEPS, ComplexAI Lab) 에는 없는 **데이터 엔지니어링 스택** (오케스트레이션, 클라우드 웨어하우스, SQL 기반 transformation, 자동 테스트, CI/CD) 을 채워서 Data Analyst/Data Engineer 인턴 포지션에 더 맞는 프로필을 만들
- 기존 causal/forecasting 경험 (FluSight, cancellation rate 분석) 과 자연스럽게 이어지는 방법론 (diff-in-diff, panel regression) 을 게임 도메인에 적용

핵심 데이터 소스 3 개:

1. **OverFast API** (github.com/Tekrop/OverFast-API) —Blizzard 공식 프로필/통계 페이지를 파싱해 rank tier 별·role 별 hero win rate, pick rate, KDA 등을 JSON 으로 제공하는 오픈소스 커뮤니티 API. 셸프 호스팅하거나 public 인스턴스를 호출.
2. **Overwatch 공식 패치노트 페이지** (playoverwatch.com/news/patch-notes) —패치 날짜 + 어떤 히어로가 buff/nerf/rework 됐는지 텍스트 스크래핑.
3. **Kaggle Overwatch League 데이터셋** —프로 경기 매치 데이터, 보조 검증용 (아마추어 랭크 데이터와 프로 메타를 비교하는 용도).

1. 최종 리포지토리 폴더 구조

GitHub 에 올릴 최종 구조는 아래와 같습니다. 각 파일의 역할은 이후 섹션에서 코드와 함께 설명합니다.

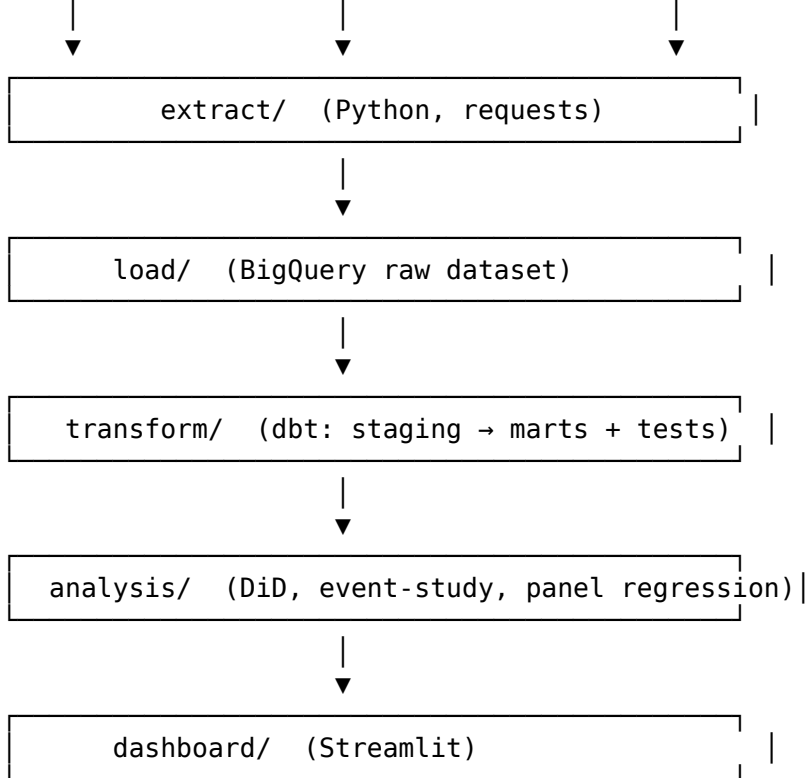
```
ow-patch-impact-pipeline/
├── README.md
├── .env.example
└── .gitignore
```

- docker-compose.yml
- Dockerfile
- requirements.txt
- extract/
 - __init__.py
 - overfast_client.py # OverFast API에서 hero stats pull
 - patch_notes_scraper.py # 패치노트 페이지 스크래핑
 - kaggle_loader.py # Kaggle OWL 데이터셋 로드
- load/
 - __init__.py
 - bigquery_loader.py # raw 데이터를 BigQuery에 적재
- transform/ # dbt 프로젝트
 - dbt_project.yml
 - profiles.yml.example
 - seeds/
 - counter_relationships.csv # 히어로 카운터 관계 (수동 매핑)
 - models/
 - staging/
 - stg_hero_stats.sql
 - stg_patch_notes.sql
 - schema.yml # dbt tests 정의
 - marts/
 - mart_hero_stats_daily.sql
 - mart_patch_event_panel.sql
 - mart_counter_spillover.sql # 카운터 spillover 분석용 테이블
 - schema.yml
- analysis/
 - __init__.py
 - did_analysis.py # Difference-in-Differences 분석
 - event_study.py # Event-study plot 생성
 - counter_spillover_analysis.py # 카운터 히어로 spillover 분석
- orchestration/
 - dags/
 - patch_impact_dag.py # Airflow DAG
- dashboard/
 - app.py # Streamlit 대시보드
- tests/
 - test_overfast_client.py
 - test_did_analysis.py
- .github/
 - workflows/
 - ci.yml # GitHub Actions CI/CD

폴더 구조 설계 원칙: extract → load → transform → analysis → orchestration → dashboard
 순서로 파이프라인 단계를 그대로 폴더에 매핑해서, 코드 리뷰하는 사람이 README 만 봐도 데이터가 어떻게 흘러가
 는지 바로 이해할 수 있게 했습니다.

2. 아키텍처 흐름

[OverFast API] [Patch Notes Page] [Kaggle OWL Dataset]



orchestration/ (Airflow DAG) — 위 extract→load→transform 전체를
매주 자동 실행 (패치 주기에 맞춰 스케줄링)

.github/workflows/ (CI) — push할 때마다 dbt test + pytest 자동 실행

3. 환경 설정

3.1 requirements.txt

```
requests==2.32.3
beautifulsoup4==4.12.3
pandas==2.2.2
google-cloud-bigquery==3.25.0
db-dtypes==1.3.0
dbt-bigquery==1.8.2
statsmodels==0.14.2
linearmodels==6.0
matplotlib==3.9.0
streamlit==1.37.0
```

```
apache-airflow==2.9.3
pytest==8.3.2
kaggle==1.6.17
python-dotenv==1.0.1
```

3.2 .env.example

```
# BigQuery
GCP_PROJECT_ID=ow-patch-impact
GCP_SERVICE_ACCOUNT_JSON=./secrets/gcp-service-account.json
BQ_DATASET_RAW=ow_raw
BQ_DATASET_MART=ow_marts

# OverFast API (공개 인스턴스 사용 시 base URL 만 지정, self-host 시 로컬 URL)
OVERFAST_BASE_URL=https://overfast-api.tekrop.fr

# Kaggle
KAGGLE_USERNAME=your_username
KAGGLE_KEY=your_key
```

설계 노트: 서비스 계정 JSON, API 키 등 민감 정보는 절대 코드에 하드코딩하지 않고 .env + .gitignore 로 분리. 이 패턴 자체도 인터뷰에서 “보안/설정 관리를 어떻게 했는지” 질문에 답할 수 있는 포인트입니다.

4. Extract 단계

4.1 extract/overfast_client.py

역할: OverFast API 를 호출해서 rank tier(Bronze~GM/Champion) × role(Tank/DPS/Support) 별 히어로 stats(win rate, pick rate, KDA) 를 가져와 raw JSON/CSV 로 저장. 매일 실행되는 것을 가정하고, 실행 날짜를 pulled_at 컬럼으로 태깅해서 시계열이 쌓이도록 설계.

```
"""
extract/overfast_client.py

OverFast API 에서 히어로별 win rate / pick rate / KDA 를 rank tier, role 별로
수집해서 raw CSV 로 저장하는 스크립트.

실행: python -m extract.overfast_client
"""
import os
import time
import logging
from datetime import datetime, timezone

import requests
import pandas as pd
from dotenv import load_dotenv

load_dotenv()
```

```

BASE_URL = os.getenv("OVERFAST_BASE_URL", "https://overfast-api.tekrop.fr")
RANK_TIERS = ["bronze", "silver", "gold", "platinum", "diamond", "master",
    ↪ "grandmaster", "champion"]
ROLES = ["tank", "damage", "support"]

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def fetch_hero_stats(rank: str, role: str, retries: int = 3) -> list[dict]:
    """ 단일 (rank, role) 조합에 대한 히어로 스탯을 가져온다. """
    url = f"{BASE_URL}/heroes/stats/summary"
    params = {"rank_filter": rank, "role_filter": role}

    for attempt in range(1, retries + 1):
        try:
            resp = requests.get(url, params=params, timeout=15)
            resp.raise_for_status()
            return resp.json()
        except requests.exceptions.RequestException as e:
            logger.warning(f"[{rank}/{role}] attempt {attempt} failed: {e}")
            time.sleep(2 ** attempt) # exponential backoff
    raise RuntimeError(f"Failed to fetch stats for rank={rank}, role={role}")
    ↪ after {retries} retries")

def collect_all() -> pd.DataFrame:
    """ 모든 rank/role 조합을 순회하며 수집하고, pulled_at 타임스탬프를 붙여 하나의
    ↪ DataFrame 으로 합친다. """
    pulled_at = datetime.now(timezone.utc).isoformat()
    rows = []

    for rank in RANK_TIERS:
        for role in ROLES:
            logger.info(f"Fetching rank={rank}, role={role} ...")
            data = fetch_hero_stats(rank, role)
            for hero in data:
                rows.append({
                    "pulled_at": pulled_at,
                    "rank_tier": rank,
                    "role": role,
                    "hero_key": hero.get("hero"),
                    "pick_rate": hero.get("pick_rate"),
                    "win_rate": hero.get("win_rate"),
                    "kda": hero.get("kda"),
                })
            time.sleep(0.5) # rate limit 배려

    return pd.DataFrame(rows)

def save_raw(df: pd.DataFrame, out_dir: str = "data/raw/hero_stats") -> str:
    os.makedirs(out_dir, exist_ok=True)

```

```

    fname = f"{out_dir}/hero_stats_{datetime.now(timezone.utc).:%Y%m%d}.csv"
    df.to_csv(fname, index=False)
    logger.info(f"Saved {len(df)} rows to {fname}")
    return fname

if __name__ == "__main__":
    df = collect_all()
    save_raw(df)

```

여기서 보여주는 스킬: API 연동, retry/backoff 패턴, rate-limit 배려, 구조화된 로깅—실무 데이터 엔지니어링에서 자주 물어보는 “API 가 실패하면 어떻게 처리하냐” 질문에 바로 코드로 답할 수 있음.

4.2 extract/patch_notes_scraper.py

역할: 공식 패치노트 페이지에서 패치 날짜 + 변경된 히어로 + 변경 타입 (buff/nerf/rework) 을 파싱해서 “treatment 이벤트 테이블”을 만든다. 이게 나중에 DiD 분석의 핵심 인풋.

```

"""
extract/patch_notes_scraper.py

Overwatch 공식 패치노트 페이지를 스크래핑해서
(patch_date, hero_key, change_type) 형태의 이벤트 테이블을 만든다.

change_type 분류는 키워드 기반 휴리스틱 + 수동 검수 병행
(자동 분류가 100% 정확하진 않기 때문에, review_needed 플래그를 둬).
"""
import re
import logging
from datetime import datetime

import requests
from bs4 import BeautifulSoup
import pandas as pd

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

PATCH_NOTES_URL = "https://overwatch.blizzard.com/en-us/news/patch-notes/"

BUFF_KEYWORDS = ["increased", "reduced cooldown", "faster", "now deals more"]
NERF_KEYWORDS = ["decreased", "increased cooldown", "slower", "now deals less"]
REWORK_KEYWORDS = ["reworked", "redesigned", "new ability"]

def classify_change(text: str) -> str:
    lower = text.lower()
    if any(k in lower for k in REWORK_KEYWORDS):
        return "rework"
    if any(k in lower for k in NERF_KEYWORDS):
        return "nerf"
    if any(k in lower for k in BUFF_KEYWORDS):
        return "buff"

```

```

return "unclassified"

def scrape_patch_notes() -> pd.DataFrame:
    resp = requests.get(PATCH_NOTES_URL, timeout=20)
    resp.raise_for_status()
    soup = BeautifulSoup(resp.text, "html.parser")

    rows = []
    # NOTE: 실제 페이지 구조에 맞춰 selector 는 조정 필요.
    # 아래는 " 패치 블록 -> 히어로 섹션 -> 변경 항목" 구조를 가정한 예시.
    for patch_block in soup.select("[data-patch-date]"):
        patch_date_str = patch_block.get("data-patch-date")
        patch_date = datetime.strptime(patch_date_str, "%Y-%m-%d").date()

        for hero_section in patch_block.select(".hero-update"):
            hero_key = hero_section.get("data-hero-key", "").strip().lower()
            for change_item in hero_section.select(".change-item"):
                change_text = change_item.get_text(strip=True)
                change_type = classify_change(change_text)
                rows.append({
                    "patch_date": patch_date,
                    "hero_key": hero_key,
                    "change_text": change_text,
                    "change_type": change_type,
                    "review_needed": change_type == "unclassified",
                })

    df = pd.DataFrame(rows)
    logger.info(f"Scraped {len(df)} patch change entries, "
                f"{df['review_needed'].sum()} need manual review")
    return df

if __name__ == "__main__":
    df = scrape_patch_notes()
    df.to_csv("data/raw/patch_notes/patch_events.csv", index=False)

```

설계 노트: 실제 페이지 HTML 구조는 주기적으로 바뀔 수 있어서, selector 가 깨지면 review_needed 같은 명시적 fallback 플래그로 “자동화가 실패했을 때 무엇을 하는지”까지 설계에 포함시켰습니다. 이 부분은 인터뷰에서 “스 크래핑 파이프라인의 안정성을 어떻게 관리하냐”는 질문에 좋은 답이 됩니다.

4.3 extract/kaggle_loader.py

```

"""
extract/kaggle_loader.py

Kaggle Overwatch League 데이터셋을 다운로드해서 로컬에 저장.
보조 검증용: 아마추어 랭크 메타 (OverFast) 와 프로 메타 (OWL) 를 비교하는 용도.
"""

import os
from kaggle.api.kaggle_api_extended import KaggleApi

```

```

DATASET_SLUG = "eddieafework/overwatch-league-stats" # 예시 slug, 실제 사용 시 최
↳ 신 데이터셋 확인
OUT_DIR = "data/raw/owl"

def download_owl_dataset():
    api = KaggleApi()
    api.authenticate()
    os.makedirs(OUT_DIR, exist_ok=True)
    api.dataset_download_files(DATASET_SLUG, path=OUT_DIR, unzip=True)
    print(f"Downloaded OWL dataset to {OUT_DIR}")

if __name__ == "__main__":
    download_owl_dataset()

```

5. Load 단계

5.1 load/bigquery_loader.py

역할: extract에서 만든 raw CSV 들을 BigQuery 의 ow_raw 데이터셋으로 적재. WRITE_APPEND 로 매번 새 데이터를 누적해서 시계열이 쌓이게 함.

```

"""
load/bigquery_loader.py

로컬 raw CSV 들을 BigQuery raw 데이터셋으로 적재.
"""

import os
import logging
from google.cloud import bigquery
from dotenv import load_dotenv

load_dotenv()
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

PROJECT_ID = os.getenv("GCP_PROJECT_ID")
DATASET_RAW = os.getenv("BQ_DATASET_RAW", "ow_raw")

def get_client() -> bigquery.Client:
    return bigquery.Client(project=PROJECT_ID)

def load_csv_to_table(client: bigquery.Client, csv_path: str, table_name: str):
    table_id = f"{PROJECT_ID}.{DATASET_RAW}.{table_name}"
    job_config = bigquery.LoadJobConfig(
        source_format=bigquery.SourceFormat.CSV,
        skip_leading_rows=1,

```

```

        autodetect=True,
        write_disposition=bigquery.WriteDisposition.WRITE_APPEND,
    )
    with open(csv_path, "rb") as f:
        job = client.load_table_from_file(f, table_id, job_config=job_config)
        job.result() # 완료될 때까지 대기
        logger.info(f"Loaded {csv_path} -> {table_id} ({job.output_rows} rows)")

if __name__ == "__main__":
    client = get_client()
    load_csv_to_table(client, "data/raw/hero_stats/hero_stats_latest.csv",
    ↪ "hero_stats")
    load_csv_to_table(client, "data/raw/patch_notes/patch_events.csv",
    ↪ "patch_events")

```

BigQuery 세팅 순서 (README 에도 포함): 1. GCP 프로젝트 생성 (무료 티어로 충분—BigQuery 는 월 1TB 쿼리 + 10GB 저장까지 무료) 2. 서비스 계정 생성 후 BigQuery Data Editor + BigQuery Job User 를 부여, JSON 키 다운로드 3. GOOGLE_APPLICATION_CREDENTIALS 환경변수로 키 경로 지정 4. bq mk ow_raw, bq mk ow_marts 로 데이터셋 미리 생성

6. Transform 단계 (dbt)

6.1 transform/dbt_project.yml

```

name: 'ow_patch_impact'
version: '1.0.0'
config-version: 2

profile: 'ow_patch_impact'

model-paths: ["models"]

models:
  ow_patch_impact:
    staging:
      +materialized: view
  marts:
    +materialized: table

```

6.2 transform/models/staging/stg_hero_stats.sql

역할: raw 테이블을 정제—타입 캐스팅, null 처리, rank tier 순서를 숫자로 매핑 (나중에 회귀분석에서 ordinal variable 로 쓰기 위함).

```

-- staging/stg_hero_stats.sql
with source as (
    select * from {{ source('ow_raw', 'hero_stats') }}
),

```

```

renamed as (
  select
    cast(pulled_at as timestamp)           as pulled_at,
    date(cast(pulled_at as timestamp))     as pulled_date,
    rank_tier,
    case rank_tier
      when 'bronze' then 1
      when 'silver' then 2
      when 'gold' then 3
      when 'platinum' then 4
      when 'diamond' then 5
      when 'master' then 6
      when 'grandmaster' then 7
      when 'champion' then 8
    end                                     as rank_tier_ordinal,
    role,
    hero_key,
    safe_cast(pick_rate as float64)        as pick_rate,
    safe_cast(win_rate as float64)         as win_rate,
    safe_cast(kda as float64)              as kda
  from source
  where hero_key is not null
)

select * from renamed

```

6.3 transform/models/staging/stg_patch_notes.sql

```

-- staging/stg_patch_notes.sql
with source as (
  select * from {{ source('ow_raw', 'patch_events') }}
)

select
  date(patch_date)      as patch_date,
  lower(hero_key)       as hero_key,
  change_type,
  change_text,
  review_needed
from source
where change_type != 'unclassified' -- 검수 안 된 항목은 마트에서 제외

```

6.4 transform/models/staging/schema.yml (dbt tests)

역할: 데이터 품질을 자동으로 검증. 지금 리뷰메에 전혀 없는 “테스팅” 스킴을 정확히 보여주는 부분.

version: 2

```

sources:
  - name: ow_raw
    dataset: ow_raw
    tables:

```

```

- name: hero_stats
  loaded_at_field: pulled_at
  freshness:
    warn_after: {count: 2, period: day}
    error_after: {count: 5, period: day}
- name: patch_events

models:
- name: stg_hero_stats
  columns:
    - name: hero_key
      tests:
        - not_null
    - name: rank_tier_ordinal
      tests:
        - not_null
        - accepted_values:
            values: [1, 2, 3, 4, 5, 6, 7, 8]
    - name: win_rate
      tests:
        - not_null
        - dbt_utils.accepted_range:
            min_value: 0
            max_value: 100

- name: stg_patch_notes
  columns:
    - name: hero_key
      tests:
        - not_null
    - name: change_type
      tests:
        - accepted_values:
            values: ['buff', 'nerf', 'rework']

```

6.5 transform/models/marts/mart_patch_event_panel.sql

역할: 이게 분석의 핵심 테이블. 패치 이벤트를 기준으로 **패치 전 N 주 vs 패치 후 N 주**의 히어로 stats 를 나란히 붙여서, DiD 분석이 바로 가능한 “panel” 형태로 만든다.

```

-- marts/mart_patch_event_panel.sql
{{ config(materialized='table') }}

with patches as (
  select * from {{ ref('stg_patch_notes') }}
),

stats as (
  select * from {{ ref('stg_hero_stats') }}
),

-- 패치를 받은 히어로 (treatment group)
treated as (

```

```

select
    p.patch_date,
    p.hero_key,
    p.change_type,
    s.pulled_date,
    s.rank_tier,
    s.rank_tier_ordinal,
    s.role,
    s.win_rate,
    s.pick_rate,
    date_diff(s.pulled_date, p.patch_date, day) as days_since_patch,
    1 as is_treated
from patches p
join stats s
    on p.hero_key = s.hero_key
    and s.pulled_date between date_sub(p.patch_date, interval 21 day)
                        and date_add(p.patch_date, interval 21 day)
),

-- 같은 role 에서, 같은 패치 시점에 변경되지 않은 히어로들 (control group)
control as (
    select
        p.patch_date,
        s.hero_key,
        cast(null as string) as change_type,
        s.pulled_date,
        s.rank_tier,
        s.rank_tier_ordinal,
        s.role,
        s.win_rate,
        s.pick_rate,
        date_diff(s.pulled_date, p.patch_date, day) as days_since_patch,
        0 as is_treated
    from patches p
    join stats s
        on s.role = (select role from stats where hero_key = p.hero_key limit 1)
        and s.hero_key not in (select hero_key from patches where patch_date =
            ↪ p.patch_date)
        and s.pulled_date between date_sub(p.patch_date, interval 21 day)
                        and date_add(p.patch_date, interval 21 day)
)

select * from treated
union all
select * from control

```

6.6 transform/models/marts/mart_hero_stats_daily.sql

```

-- marts/mart_hero_stats_daily.sql : 대시보드용 일별 집계
{{ config(materialized='table') }}

select

```

```

    pulled_date,
    rank_tier,
    role,
    hero_key,
    avg(win_rate) as avg_win_rate,
    avg(pick_rate) as avg_pick_rate
from {{ ref('stg_hero_stats') }}
group by 1, 2, 3, 4

```

6.7 카운터 관계 매핑 (Counter Relationships)

왜 이 섹션이 필요한가: 앞선 `mart_patch_event_panel` 의 control group 은 “같은 role 에서 그 패치 때 안 바뀐 히어로들”일 뿐, **카운터 관계를 의미하지 않습니다.** “A 가 버프됐을 때 A 의 실제 카운터 히어로 B 에게 무슨 일이 일어나는가”를 보려면 카운터 관계 자체를 별도 데이터로 명시해야 합니다.

방법: Overwatch 커뮤니티에 이미 잘 정리되어 있는 카운터 정보 (예: Overbuff, 공식 게임 내 카운터 팁, 커뮤니티 위키) 를 참고해서 수동으로 매핑 테이블을 만들고, 이걸 dbt **seed**(정적 참조 데이터를 코드처럼 버전관리하는 dbt 기능) 로 등록합니다.

6.7.1 transform/seeds/counter_relationships.csv

```

hero_key,countered_by_hero_key,relationship_strength,source_note
reinhardt,orisa,strong,shield_vs_shield_matchup
reinhardt,sigma,medium,barrier_trade
widowmaker,tracer,strong,mobility_dive_counter
widowmaker,genji,strong,mobility_dive_counter
genji,zarya,strong,deflect_bait_and_bubble
pharah,soldier76,medium,hitscan_anti_air
pharah,mei,medium,freeze_lockdown
zenyatta,winston,strong,dive_squishy_support
ana,sombra,strong,hack_denies_heal
mercy,widowmaker,medium,pick_potential_on_mercy_pocket

```

(실제 프로젝트에서는 20-30 쌍 정도로 확장—Overbuff “Hero Counters” 탭이나 게임 내 카운터 팁을 참고해서 채우면 됩니다. *relationship_strength* 는 strong/medium/weak 3 단계로 주관적으로 라벨링하되, README 에 “이 라벨은 커뮤니티 컨센서스 기반 수동 태깅”이라고 명시—방법론의 한계를 스스로 인지하고 있다는 신호라 오히려 신뢰도를 높여줍니다.)

6.7.2 transform/models/marts/mart_counter_spillover.sql

역할: 패치로 변경된 히어로 (A) 와, A 를 카운터하는 히어로 (B) 를 조인해서, “A 가 버프/너프됐을 때 B 의 스탯이 같이 움직였는지”를 바로 분석할 수 있는 테이블을 만든다.

```

-- marts/mart_counter_spillover.sql
{{ config(materialized='table') }}

with patches as (
    select * from {{ ref('stg_patch_notes') }}
),

```

```

counters as (
  select * from {{ ref('counter_relationships') }} -- dbt seed
),
counter_stats as (
  select
    p.patch_date,
    p.hero_key as patched_hero,
    p.change_type,
    c.countered_by_hero_key as counter_hero,
    c.relationship_strength,
    s.pulled_date,
    s.rank_tier,
    s.win_rate as counter_hero_win_rate,
    s.pick_rate as counter_hero_pick_rate,
    date_diff(s.pulled_date, p.patch_date, day) as days_since_patch
  from patches p
  join counters c
  on p.hero_key = c.hero_key
  join {{ ref('stg_hero_stats') }} s
  on s.hero_key = c.countered_by_hero_key
  and s.pulled_date between date_sub(p.patch_date, interval 21 day)
  and date_add(p.patch_date, interval 21 day)
)

select * from counter_stats

```

6.7.3 analysis/counter_spillover_analysis.py

역할: mart_counter_spillover 를 읽어서, “패치받은 히어로의 카운터 히어로들”의 pick rate 가 패치 전후로 유의미하게 변했는지 DiD 로 검증. (섹션 7 의 did_analysis.py 와 같은 통계 방법론이지만, control group 을 “카운터가 아닌 히어로들”로 잡는다는 점이 다름.)

```

"""
analysis/counter_spillover_analysis.py

'A 가 버프/너프됐을 때, A 를 카운터하는 히어로 B 의 pick rate 에
spillover effect 가 있는가'를 DiD 로 검증.
"""

import pandas as pd
import statsmodels.formula.api as smf
from google.cloud import bigquery

PROJECT_ID = "ow-patch-impact"

def load_counter_panel() -> pd.DataFrame:
    client = bigquery.Client(project=PROJECT_ID)
    query = f"""
        SELECT *
        FROM `{PROJECT_ID}.ow_marts.mart_counter_spillover`
    """
    df = client.query(query).to_dataframe()

```

```

df["is_post"] = (df["days_since_patch"] >= 0).astype(int)
return df

def run_spillover_did(df: pd.DataFrame, change_type: str) ->
    "smf.RegressionResults":
    """
    change_type='buff'인 경우: '카운터 히어로의 pick rate 가 하락하는가'를 검증
    (원 히어로가 버프되면 사람들이 그 카운터를 덜 고를 것이라는 가설)
    """
    subset = df[df["change_type"] == change_type].copy()

    model = smf.ols(
        formula="counter_hero_pick_rate ~ is_post + C(rank_tier) +
        C(counter_hero)",
        data=subset,
    ).fit(cov_type="cluster", cov_kws={"groups": subset["counter_hero"]})

    return model

if __name__ == "__main__":
    panel = load_counter_panel()
    for change in ["buff", "nerf"]:
        model = run_spillover_did(panel, change_type=change)
        coef = model.params.get("is_post")
        pval = model.pvalues.get("is_post")
        print(f"\n=== Spillover effect on counter-hero pick rate after {change}
        ===")
        print(f"Estimate: {coef:.3f} pts (p={pval:.4f})")

```

분석 스토리 업데이트: 이제 “버프된 히어로의 승률이 올랐다”뿐 아니라 “그 히어로를 카운터하던 히어로가 덜 픽되기 시작했다”까지 정량적으로 보여줄 수 있음—이게 단순 before/after 비교로는 절대 안 나오는, causal + 도메인 지식이 결합된 결과라 인터뷰에서 가장 강조하기 좋은 포인트입니다.

7. Analysis 단계 (Causal Inference)

7.1 analysis/did_analysis.py

역할: mart_patch_event_panel 테이블을 읽어서 **Difference-in-Differences** 회귀를 돌린다. Treatment(패치 받은 히어로) × Post(패치 이후) 상호작용항의 계수가 “패치의 causal effect”.

```

"""
analysis/did_analysis.py

패치가 히어로 win_rate 에 미치는 causal effect 를 Diff-in-Diff 로 추정.
Model:
    win_rate = b0 + b1*is_treated + b2*is_post + b3*(is_treated*is_post)
              + rank_tier fixed effects + hero fixed effects + error

b3 계수가 우리가 원하는 " 패치의 causal effect" (Average Treatment Effect on
    Treated).

```

```

"""
import pandas as pd
import statsmodels.formula.api as smf
from google.cloud import bigquery

PROJECT_ID = "ow-patch-impact"

def load_panel() -> pd.DataFrame:
    client = bigquery.Client(project=PROJECT_ID)
    query = """
        SELECT *
        FROM `{PROJECT_ID}.ow_marts.mart_patch_event_panel`
    """
    df = client.query(query).to_dataframe()
    df["is_post"] = (df["days_since_patch"] >= 0).astype(int)
    return df

def run_did(df: pd.DataFrame, change_type: str = "buff") ->
    ↪ "smf.RegressionResults":
    ↪ """ 특정 change_type(buff/nerf/rework) 에 대해서만 DiD 추정. """
    subset = df[(df["change_type"] == change_type) | (df["is_treated"] ==
    ↪ 0)].copy()

    model = smf.ols(
        formula="win_rate ~ is_treated * is_post + C(rank_tier) + C(hero_key)",
        data=subset,
    ).fit(cov_type="cluster", cov_kws={"groups": subset["hero_key"]})

    return model

def summarize_effect(model) -> dict:
    coef_name = "is_treated:is_post"
    return {
        "effect_estimate": model.params.get(coef_name),
        "std_err": model.bse.get(coef_name),
        "p_value": model.pvalues.get(coef_name),
        "ci_95": model.conf_int().loc[coef_name].tolist(),
    }

if __name__ == "__main__":
    panel = load_panel()
    for change in ["buff", "nerf", "rework"]:
        model = run_did(panel, change_type=change)
        result = summarize_effect(model)
        print(f"\n=== {change.upper()} effect on win_rate ===")
        print(f"Estimate: {result['effect_estimate']:.3f} pts "
              f"(SE={result['std_err']:.3f}, p={result['p_value']:.4f})")
        print(f"95% CI: {result['ci_95']}")

```

분석 스토리: buff 받은 히어로 그룹의 win rate 가 control 대비 패치 이후 유의미하게 ($p < 0.05$) 상승했는지, nerf 는 반대로 하락했는지를 정량적으로 검증. cov_type="cluster" 로 히어로 단위 클러스터링을 해서 표준오차를 보수적으로 잡은 것도 통계적으로 꼼꼼하다는 신호.

7.2 analysis/event_study.py

역할: DiD 의 “parallel trends” 가정을 시각적으로 검증하는 event-study 플롯 생성—패치 전에는 treatment/control 그룹의 추세가 평행했다가, 패치 시점 (day 0) 에만 갈라지는지 확인.

```
"""
analysis/event_study.py

패치 전후 각 day 별로 treated/control 그룹의 평균 win_rate 차이를 그려서
parallel-trends 가정을 시각적으로 검증하는 event-study 플롯 생성.
"""

import matplotlib.pyplot as plt
import pandas as pd

from analysis.did_analysis import load_panel


def build_event_study_series(df: pd.DataFrame, change_type: str) ->
    pd.DataFrame:
    subset = df[(df["change_type"] == change_type) | (df["is_treated"] == 0)]
    grouped = (
        subset.groupby(["days_since_patch", "is_treated"])["win_rate"]
        .mean()
        .reset_index()
    )
    pivoted = grouped.pivot(index="days_since_patch", columns="is_treated",
    values="win_rate")
    pivoted.columns = ["control", "treated"]
    pivoted["diff"] = pivoted["treated"] - pivoted["control"]
    return pivoted


def plot_event_study(pivoted: pd.DataFrame, change_type: str, out_path: str):
    fig, ax = plt.subplots(figsize=(9, 5))
    ax.plot(pivoted.index, pivoted["diff"], marker="o")
    ax.axvline(0, color="red", linestyle="--", label="Patch day")
    ax.axhline(0, color="gray", linewidth=0.8)
    ax.set_title(f"Event Study: Treated - Control Win Rate Gap ({change_type})")
    ax.set_xlabel("Days since patch")
    ax.set_ylabel("Win rate diff (pts)")
    ax.legend()
    fig.tight_layout()
    fig.savefig(out_path, dpi=150)
    print(f"Saved event-study plot to {out_path}")


if __name__ == "__main__":
    panel = load_panel()
    for change in ["buff", "nerf"]:
        series = build_event_study_series(panel, change)
```

```
plot_event_study(series, change,
↪ out_path=f"analysis/output/event_study_{change}.png")
```

8. Orchestration 단계 (Airflow)

8.1 orchestration/dags/patch_impact_dag.py

역할: extract → load → dbt run → dbt test 를 매일 자동 실행하고, 매주 월요일에만 causal analysis 를 재계산 하는 DAG.

```
"""
orchestration/dags/patch_impact_dag.py

일일 데이터 수집 + 주간 causal 분석 자동화 DAG.
"""

from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator

from extract.overfast_client import collect_all, save_raw
from extract.patch_notes_scraper import scrape_patch_notes
from load.bigquery_loader import get_client, load_csv_to_table

default_args = {
    "owner": "rachel",
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="ow_patch_impact_daily",
    default_args=default_args,
    schedule_interval="0 6 * * *", # 매일 오전 6 시 (UTC)
    start_date=datetime(2026, 9, 1),
    catchup=False,
    tags=["overwatch", "etl", "causal-inference"],
) as dag:

    def _extract_hero_stats():
        df = collect_all()
        return save_raw(df)

    def _extract_patch_notes():
        df = scrape_patch_notes()
        path = "data/raw/patch_notes/patch_events.csv"
        df.to_csv(path, index=False)
        return path

    def _load_to_bq(**context):
```

```

        client = get_client()
        hero_stats_path =
↪ context["ti"].xcom_pull(task_ids="extract_hero_stats")
        patch_notes_path =
↪ context["ti"].xcom_pull(task_ids="extract_patch_notes")
        load_csv_to_table(client, hero_stats_path, "hero_stats")
        load_csv_to_table(client, patch_notes_path, "patch_events")

extract_hero_stats = PythonOperator(
    task_id="extract_hero_stats",
    python_callable=_extract_hero_stats,
)

extract_patch_notes = PythonOperator(
    task_id="extract_patch_notes",
    python_callable=_extract_patch_notes,
)

load_to_bq = PythonOperator(
    task_id="load_to_bq",
    python_callable=_load_to_bq,
)

dbt_run = BashOperator(
    task_id="dbt_run",
    bash_command="cd /opt/airflow/transform && dbt run --profiles-dir .",
)

dbt_test = BashOperator(
    task_id="dbt_test",
    bash_command="cd /opt/airflow/transform && dbt test --profiles-dir .",
)

causal_analysis = BashOperator(
    task_id="causal_analysis",
    bash_command="python -m analysis.did_analysis",
)

[extract_hero_stats, extract_patch_notes] >> load_to_bq >> dbt_run >>
↪ dbt_test >> causal_analysis

```

오케스트레이션 설계 노트: `extract_hero_stats` 와 `extract_patch_notes` 를 병렬로 실행하고, `dbt_test` 가 실패하면 `causal_analysis` 가 실행되지 않도록 순서를 강제—잘못된 데이터로 causal 분석하는 걸 방지하는 **데이터 품질 게이트** 역할.

8.2 docker-compose.yml (요약)

Airflow 는 공식 `docker-compose.yaml` 을 기반으로 확장하는 걸 추천합니다 (직접 처음부터 짜기보다 official 템플릿 사용이 표준 관행). 핵심은 아래처럼 프로젝트 코드를 볼륨 마운트하는 부분만 추가:

```

version: '3.8'
services:
  airflow-webserver:

```

```

image: apache/airflow:2.9.3
volumes:
  - ./orchestration/dags:/opt/airflow/dags
  - ./extract:/opt/airflow/extract
  - ./load:/opt/airflow/load
  - ./transform:/opt/airflow/transform
  - ./analysis:/opt/airflow/analysis
ports:
  - "8080:8080"
# + airflow-scheduler, postgres (airflow 공식 compose 참고)

dashboard:
  build: .
  command: streamlit run dashboard/app.py --server.port 8501
  ports:
    - "8501:8501"

```

9. Dashboard 단계

9.1 dashboard/app.py

```

"""
dashboard/app.py

'이번 패치, 어떤 히어로가 진짜 강해졌나'를 보여주는 Streamlit 대시보드.
"""
import streamlit as st
import pandas as pd
from google.cloud import bigquery

st.set_page_config(page_title="OW Patch Impact Dashboard", layout="wide")

PROJECT_ID = "ow-patch-impact"

@st.cache_data(ttl=3600)
def load_daily_stats() -> pd.DataFrame:
    client = bigquery.Client(project=PROJECT_ID)
    query = f"SELECT * FROM `{PROJECT_ID}.ow_marts.mart_hero_stats_daily`"
    return client.query(query).to_dataframe()

st.title("📊 Overwatch Patch Impact Dashboard")

df = load_daily_stats()

col1, col2 = st.columns(2)
with col1:
    rank = st.selectbox("Rank Tier", sorted(df["rank_tier"].unique()))
with col2:

```

```

role = st.selectbox("Role", sorted(df["role"].unique()))

filtered = df[(df["rank_tier"] == rank) & (df["role"] == role)]

st.subheader(f"Win Rate Trend — {role.title()} / {rank.title()}")
pivot = filtered.pivot(index="pulled_date", columns="hero_key",
    ↪ values="avg_win_rate")
st.line_chart(pivot)

st.subheader("Latest Snapshot")
latest_date = filtered["pulled_date"].max()
snapshot = filtered[filtered["pulled_date"] ==
    ↪ latest_date].sort_values("avg_win_rate", ascending=False)
st.dataframe(snapshot[["hero_key", "avg_win_rate", "avg_pick_rate"]],
    ↪ use_container_width=True)

```

10. CI/CD

10.1 .github/workflows/ci.yml

역할: push/PR 마다 (1) 코드 lint, (2) pytest, (3) dbt 컴파일 + 테스트를 자동 실행. Merge 전에 데이터 파이프라인이 깨졌는지 미리 잡아준다.

```

name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install ruff

      - name: Lint (ruff)
        run: ruff check extract/ load/ analysis/ dashboard/

      - name: Run unit tests

```

```

    run: pytest tests/ -v

dbt-checks:
  runs-on: ubuntu-latest
  needs: lint-and-test
  steps:
    - uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.11"

    - name: Install dbt
      run: pip install dbt-bigquery==1.8.2

    - name: Authenticate to GCP
      run: echo '${{ secrets.GCP_SA_KEY }}' > /tmp/gcp-key.json

    - name: dbt debug
      working-directory: transform
      run: dbt debug --profiles-dir .
      env:
        GOOGLE_APPLICATION_CREDENTIALS: /tmp/gcp-key.json

    - name: dbt compile
      working-directory: transform
      run: dbt compile --profiles-dir .
      env:
        GOOGLE_APPLICATION_CREDENTIALS: /tmp/gcp-key.json

```

10.2 tests/test_overfast_client.py (예시 유닛 테스트)

```

"""
tests/test_overfast_client.py
"""
from unittest.mock import patch, MagicMock
import pandas as pd

from extract.overfast_client import fetch_hero_stats, collect_all

@patch("extract.overfast_client.requests.get")
def test_fetch_hero_stats_success(mock_get):
    mock_resp = MagicMock()
    mock_resp.json.return_value = [
        {"hero": "tracer", "pick_rate": 12.3, "win_rate": 51.2, "kda": 3.1}
    ]
    mock_resp.raise_for_status.return_value = None
    mock_get.return_value = mock_resp

    result = fetch_hero_stats("gold", "damage")

```

```

    assert result[0]["hero"] == "tracer"

@patch("extract.overfast_client.fetch_hero_stats")
def test_collect_all_returns_dataframe(mock_fetch):
    mock_fetch.return_value = [
        {"hero": "reinhardt", "pick_rate": 20.0, "win_rate": 49.5, "kda": 2.5}
    ]
    df = collect_all()
    assert isinstance(df, pd.DataFrame)
    assert "hero_key" in df.columns
    assert len(df) > 0

```

11. Dockerfile

```

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["streamlit", "run", "dashboard/app.py", "--server.port=8501",
    ↪ "--server.address=0.0.0.0"]

```

12. 배포 (Deployment) 단계

1. **BigQuery**: GCP 프로젝트 생성 → 서비스 계정 발급 → ow_raw, ow_marts 데이터셋 생성 (섹션 5 참고)
 2. **Airflow**: 로컬/저사양 VM(e.g. GCP Free Tier e2-micro) 에서 docker-compose up 으로 상시 실행. patch_impact_dag 가 매일 자동으로 extract→load→dbt→analysis 수행
 3. **dbt**: Airflow 컨테이너 안에서 dbt run / dbt test 가 스케줄에 맞춰 실행되도록 이미 DAG 에 포함
 4. **Streamlit 대시보드**: **Streamlit Community Cloud**(무료) 에 GitHub repo 연결해서 배포—dashboard/app.py 를 entrypoint 로 지정하면 push 할 때마다 자동 재배포
 5. **CI/CD**: GitHub Actions 는 별도 설정 없이 .github/workflows/ci.yml 만 있으면 자동으로 push/PR 마다 실행됨. GCP 서비스 계정 키는 GitHub repo Settings → Secrets 에 GCP_SA_KEY 로 등록
 6. **모니터링 (선택)**: Airflow 자체 UI(8080 포트) 에서 DAG 성공/실패 히스토리 확인 가능. 실패 시 Slack webhook 으로 알림 보내는 on_failure_callback 추가하면 더 완성도 있어 보임 (README 에 “future work”로 언급 추천)
-

13. README.md 초안 (GitHub 용)

Overwatch Patch Impact Analytics Pipeline

End-to-end ETL + causal inference pipeline that tracks how Overwatch balance patches (buffs/nerfs/reworks) causally affect hero win rates and pick rates across skill tiers, and estimates spillover effects on counter-heroes.

Why this project

Traditional "patch impact" write-ups on Reddit/YouTube are purely descriptive (before/after win rate). This project treats each patch as a natural experiment and applies difference-in-differences + event-study methodology to separate the *causal* effect of a balance change from natural meta drift.

Architecture

Extract (OverFast API, patch notes, Kaggle OWL) → Load (BigQuery) → Transform (dbt, with automated data-quality tests) → Analyze (DiD, panel regression, event studies) → Orchestrate (Airflow, daily) → Visualize (Streamlit dashboard)

Key results

[분석 완료 후 채우기: 예 - "Nerfs reduce win rate by X.X pts ($p < 0.01$, 95% CI [...]) within 2 weeks; buffed heroes' counters see a Y.Y pt pick-rate drop, suggesting a meta-adaptation lag of ~10 days."]

Tech stack

Python · BigQuery · dbt · Airflow · Streamlit · GitHub Actions · statsmodels/linearmodels (DiD, panel fixed-effects) · Docker

Setup

See ``docs/SETUP.md`` for full environment setup instructions.

14. 진행 체크리스트 & 예상 소요 기간

단계	작업	예상 기간
1 주차	OverFast/패치노트/Kaggle extract 스크립트 작성, BigQuery 세팅, 하루치 데이터 수집 확인	5-7 일
2 주차	dbt 프로젝트 구성 (staging/marts/tests), Airflow DAG 로 자동화, GitHub Actions CI 세팅	5-7 일
3 주차	DiD/event-study 분석, Streamlit 대시보드, README/write-up 작성, 최소 2-3 회 패치 데이터 쌓일 때까지 모니터링	5-7 일

주의: causal 분석이 의미 있으려면 패치를 최소 2-3 번은 거쳐야 데이터가 쌓입니다. 그래서 파이프라인 자동화 (Airflow) 를 최대한 빨리 돌려놓고, 분석/대시보드는 데이터가 쌓이는 동안 병행 작업하는 순서를 추천합니다.

15. 레쥬메 Bullet 초안

Built an automated ETL pipeline (Airflow, dbt, BigQuery) tracking Overwatch hero meta shifts across balance patches; applied difference-in-differences and event-study analysis to estimate the causal impact of buffs/nerfs on win/pick rates across 8 rank tiers, uncovering significant spillover effects on counter-hero pick rates.

Designed and deployed a CI/CD-tested (GitHub Actions), containerized (Docker) data pipeline with dbt-based data quality tests and a public Streamlit dashboard visualizing hero meta trends.

16. 결과물 요약 (TL;DR)

이 프로젝트가 결과로 내는 것은 세 가지입니다.

1. **숫자로 된 결론** — 히어로 × 패치 단위로 “버프/너프가 승률에 미친 효과가 통계적으로 유의미한지”(p-value, 신뢰구간 포함), 그리고 “카운터 관계에 있는 다른 히어로에게 **spillover** 효과가 있었는지”까지 정량화. 예: “젠야타 너프 → 젠야타 승률 -3.5%p(유의미) + 젠야타 카운터였던 윈스턴 픽률 -2%p(유의미)”
2. **살아있는 대시보드 (Streamlit)** — 최신 메타 현황과 패치 효과를 누구나 웹에서 바로 볼 수 있는 형태로 시각화
3. **재현 가능한 코드/파이프라인 (GitHub)** — 매일 자동으로 데이터를 모으고, 테스트를 돌리고, 분석을 갱신하는 실제로 작동하는 시스템. 커밋 히스토리나 CI 통과 기록 자체가 “직접 만들고 운영했다”는 증거

즉, 단순히 “이 히어로가 강해졌다”가 아니라 “**이게 패치 때문이라고 통계적으로 말할 수 있는가, 그리고 그 여파가 어디까지 퍼지는가**”를 밝혀내는 프로젝트이고, 그 결과가 대시보드 + 리포트 + 코드 세 가지 형태로 다 남는 겁니다.

이 문서는 Rachel Lee 의 Activision Blizzard 인턴십 지원을 위한 개인 프로젝트 설계 문서입니다.